



Radiance and Muon: A Parameterized, Silicon-Focused SIMT Core for Exploring Accelerator-Rich GPUs

Hansung Kim, Richard Yan, Shashank Anand, Joshua You, Chris Fletcher, Sophia Shao
SLICE Winter Retreat | Jan 12, 2026



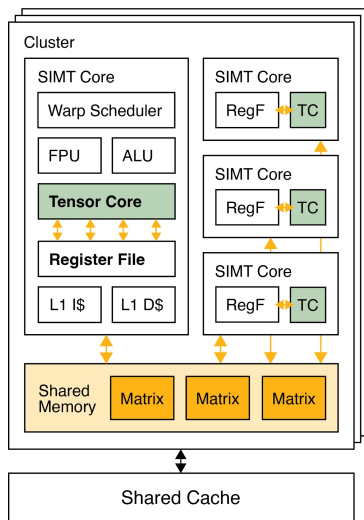
Virgo to Radiance: Heterogeneous GPUs

Heterogeneity in Modern GPUs

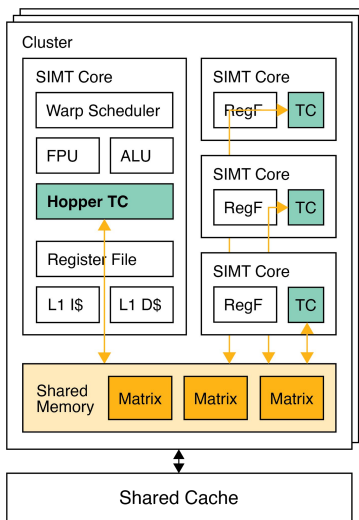
Compute demand from AI → **① Specialized matrix units** tightly-coupled to the SIMT core

Larger matrix units ← **② Longer latency hard to hide by multithreading** → **Asynchrony**

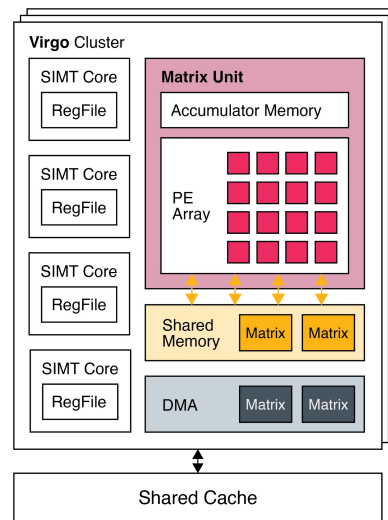
③ Register pressure and physical design → **Decoupling from core**



① Specialized Units
e.g. Volta Tensor Cores



② Asynchronous Units
e.g. Hopper Tensor Cores



③ Asynchronous-Decoupled Units
e.g. Virgo, NVIDIA RT Cores,
Hopper TMA, Blackwell TMEM



Radiance GPU

Radiance is an SoC GPU that advances Virgo to tackle GPU-based accelerator orchestration challenges with:

Neutrino Management Unit

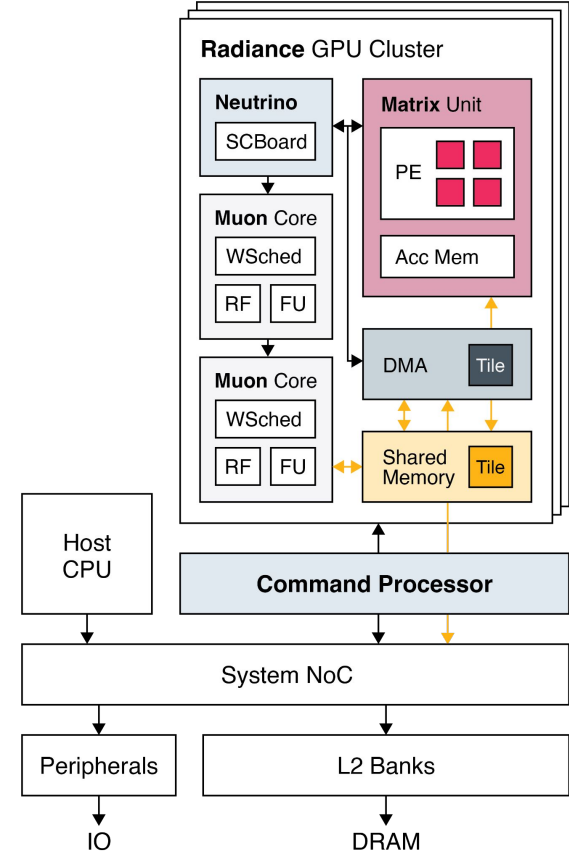
Intra-kernel fusion with Tile-level task scheduling
Lead: Richard Yan

Muon SIMT Core

High IPC and representative ASIC PPA
Lead: Hansung Kim

Command Processor

Kernel-level task scheduling / CPU-GPU interface
Lead: Shashank Anand





Muon SIMT Core Design



Modern GPU Kernel Requirements

❶ SotA GPU kernels use a lot of **registers**

- Kernels do aggressive loop-unrolling and reg-tiling to reduce instruction stalls
- Fused mega-kernels increase register pressure (FA-3: 168 regs out of 255)

❷ **Instruction-level parallelism** is important, not just thread-LP

- Register pressure limits # of threads (*warps*) spawned in core
- Core relies on ILP for latency-hiding, as TLP (*occupancy*) becomes scarce
- Compiler-driven + HW-driven inst scheduling must work hand-in-hand



Modern GPU Kernel Requirements

- ③ **Accelerators** + **warp specialization** makes occupancy scarcer
- MIMD-style producer-consumer programming in SIMT
 - HW accelerators (Tensor Cores / DMA) reserve *manager warp* slots
 - Fewer warp slots & higher reg pressure for *consumer warps*



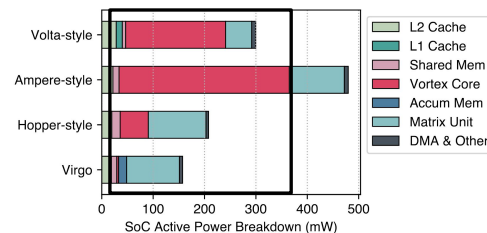
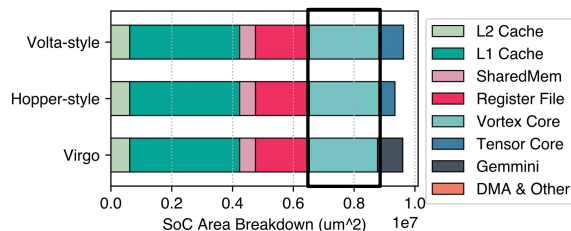
Limitations of Existing SIMT Core IP

Vortex^[1] SIMT core does not allow us to write performant GPU kernels:

- ❶ Small register space; Excessive **register spills to memory** (-10~40% GEMM perf)
- ❷ Simple pipeline, low IPC; limits **Tensor FLOPS utilization**
- ❸ **Fixed warp occupancy**

FPGA-optimized; Skews area/power tradeoffs on ASIC

→ *No academic SIMT designs target ASIC* (Ventus, Nyuzi, Simty, MIAOW)





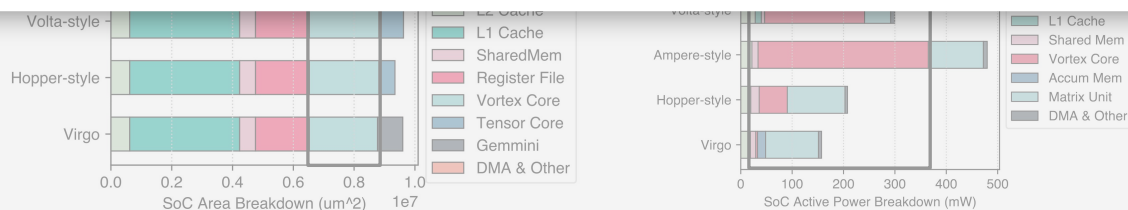
Limitations of Existing SIMT Core IP

Vortex^[1] SIMT core does not allow us to write performant GPU kernels:

- ① Small register space; Excessive **register spills to memory** (-10~40% GEMM perf)
- ② Simple pipeline, low IPC; limits **Tensor FLOPS utilization**

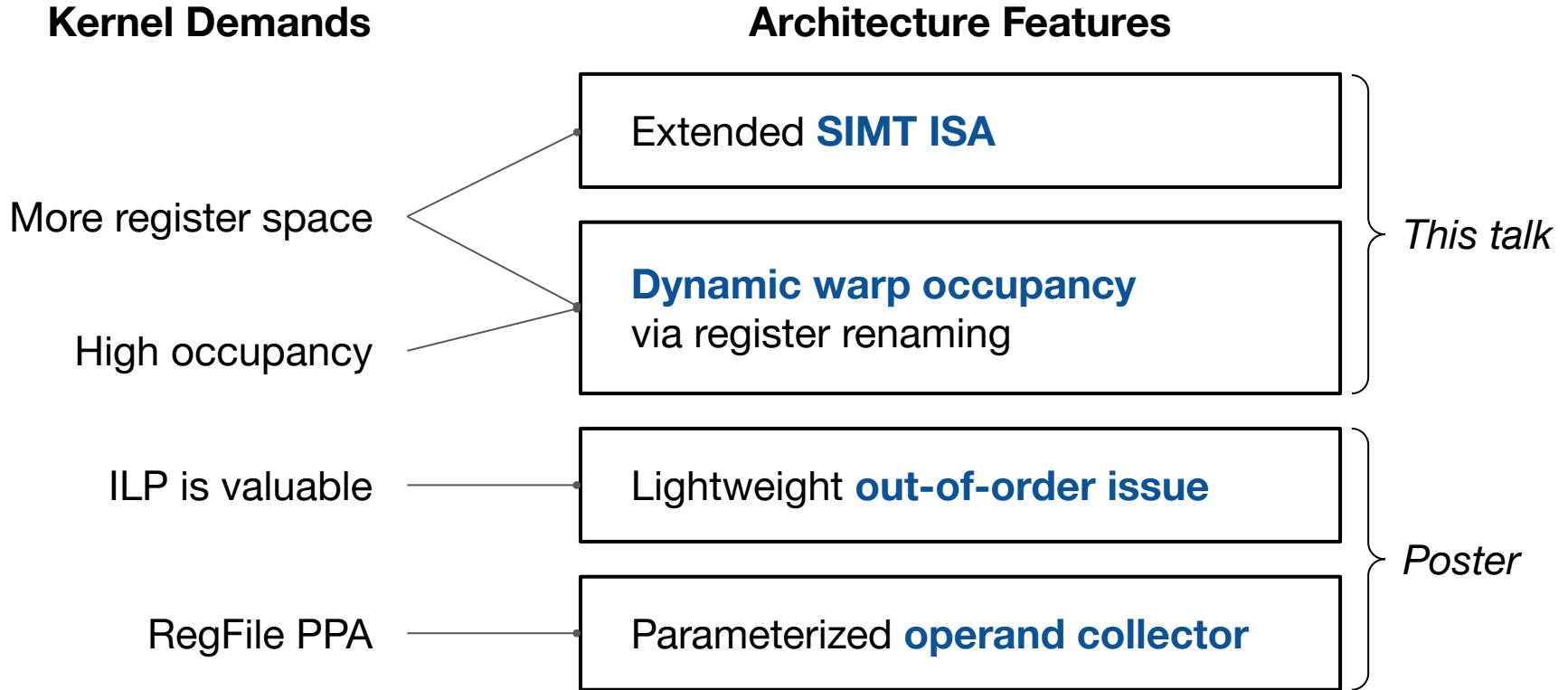
To enable research on heterogeneous GPUs,
we need a new **open-source SIMT core design** that:

- ① Architecturally enables **modern GPU kernel optimizations**, and
- ② **Targets ASIC** as the first-class platform





Muon Architectural Features



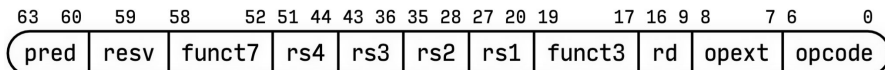


Muon GPU ISA

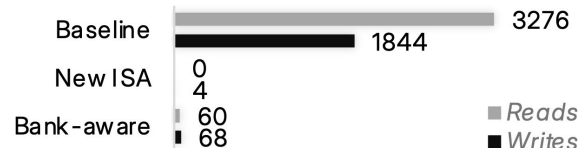
Challenge: Vortex ISA has **small register space** (32 INT + 32 FP per thread)

Solution: **Widen ISA** encoding to reserve **256 registers per thread**

- RISC-V 32 bit → **64 bit** instruction length
- **Unify** integer and floating point registers (Zfinx, Zhinx)
- Nearly eliminates **register spills** for a SIMT-only GEMM kernel with fully-unrolled inner-K-loop



Stack accesses (spills + saves):





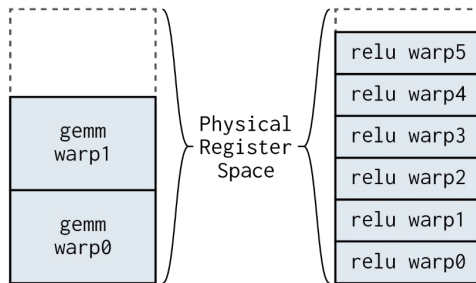
Dynamic Warp Occupancy

To have many regs in ISA, need **HW occupancy management**

- 256 registers for every thread need large physical memory
- But, *not all kernels are created the same*
- Muon adjusts **# of spawned warps** = “*occupancy*” depending on kernel’s register usage
 - Enables **ILP** (ex. *gemm*) $\leftrightarrow$ **TLP** (ex. *relu*) tradeoff for programmers

```
gemm() {  
  // k-loop  
  // smem load (...)  
  fmadd ft1, -, ft1  
  fmadd ft2, -, ft2  
  fmadd ft3, -, ft3  
  fmadd ft4, -, ft4  
  fmadd ft5, -, ft5  
  fmadd ft6, -, ft6  
  bnez t0, k-loop  
  // gmem store (...)  
}
```

Low-occupancy
Kernel



```
relu() {  
  lw    ft1, 0(a0)  
  fmax.s ft1, ft1, ft0  
  sw    ft1, 0(a0)  
}
```

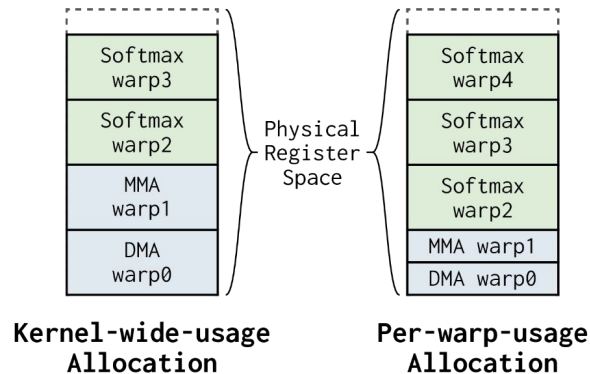
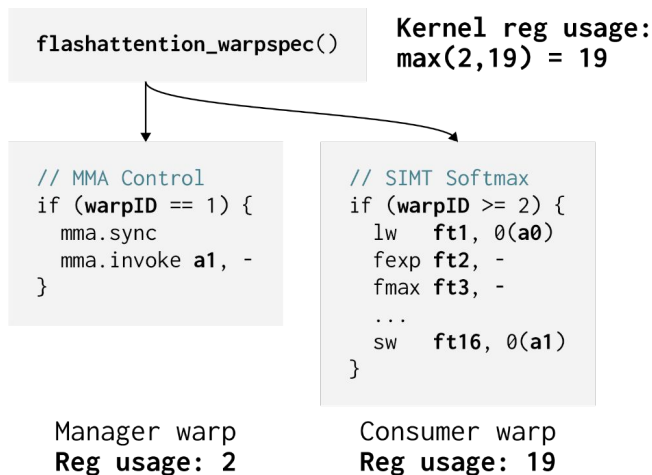
High-occupancy
Kernel



Warp Specialization

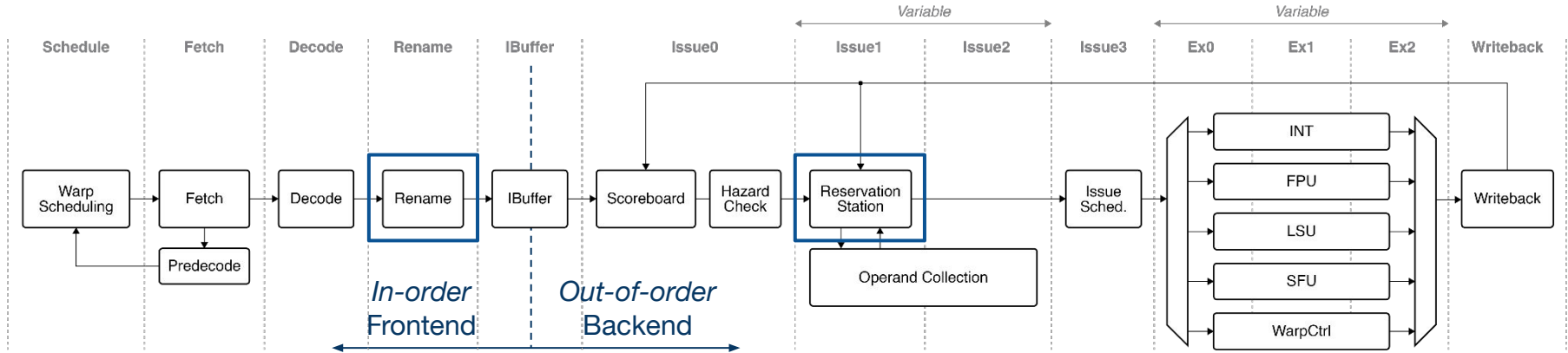
Accelerators + warp specialization call for **smarter register allocation**

- **Accel-manager** warps use fewer regs; **Consumer** warps use more regs
- Allocating **kernel-wide** usage to every warp = $\max(\text{all warps})$ causes under-utilization
- Allocating **per-warp** usage can increase occupancy → **Register renamer**





Muon Pipeline Architecture



- **In-order** fetch, decode and renaming
- **Out-of-order** issue, execute and writeback
- **Renamer** enables tight per-warp reg allocation and lowers register pressure
- **Reservation station** does OoO issue to maximize ILP for low-occupancy kernels

Details in poster!



Memory & SoC Integration



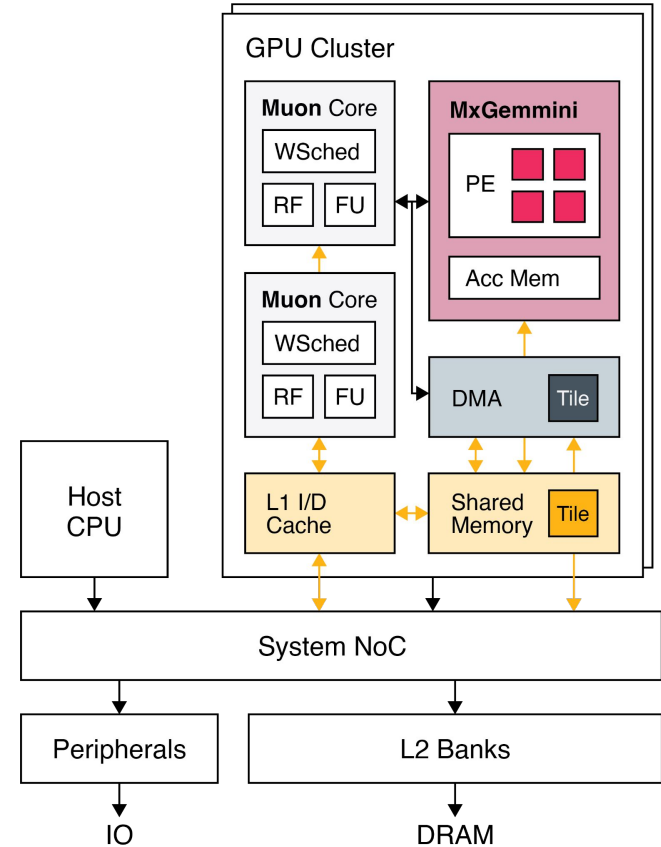
Tapeout Plans

Silicon evaluation is a key goal for Muon

Tapeout of an SoC-based Radiance GPU integrating Muon with Virgo-style mixed-precision Tensor Core

- 4 Muon SIMT cores
- 2 Gemmini-Mx (FP4/FP6/FP8)
- TSMC 16nm
- Target application: LLM Decoder (Attention + FFN)

More in the next talk!



Integration: Scope of Discussion



**Global
Memory**

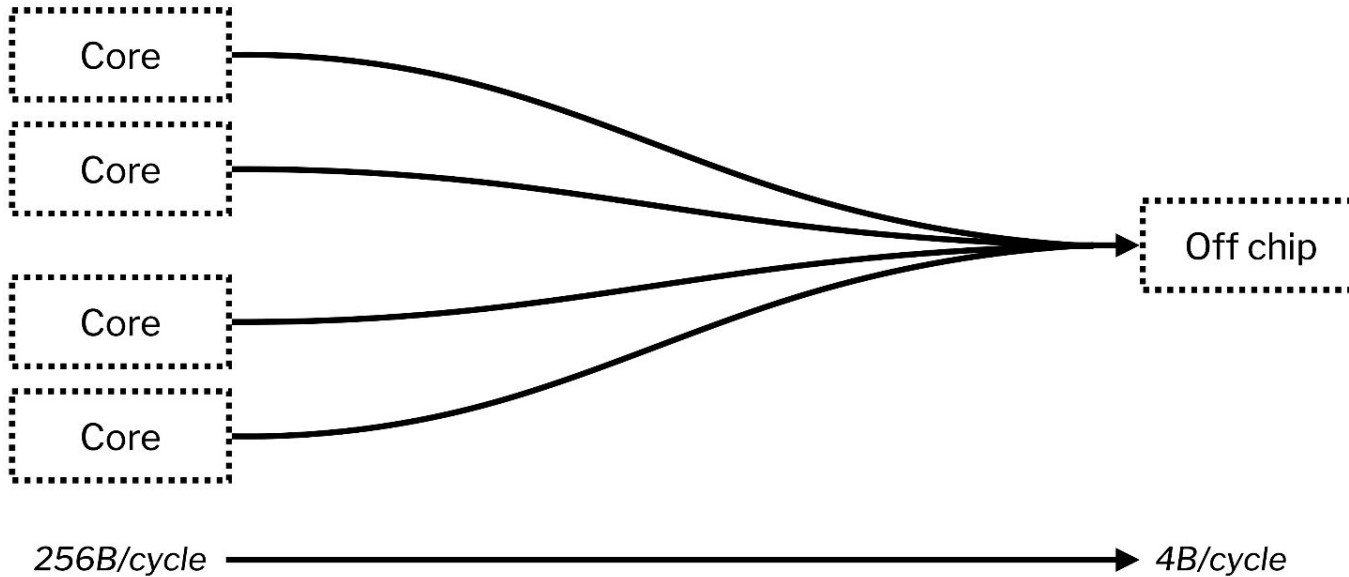
**Cluster
Shared
Memory**

**Data &
Control
Fabric**

Global Memory



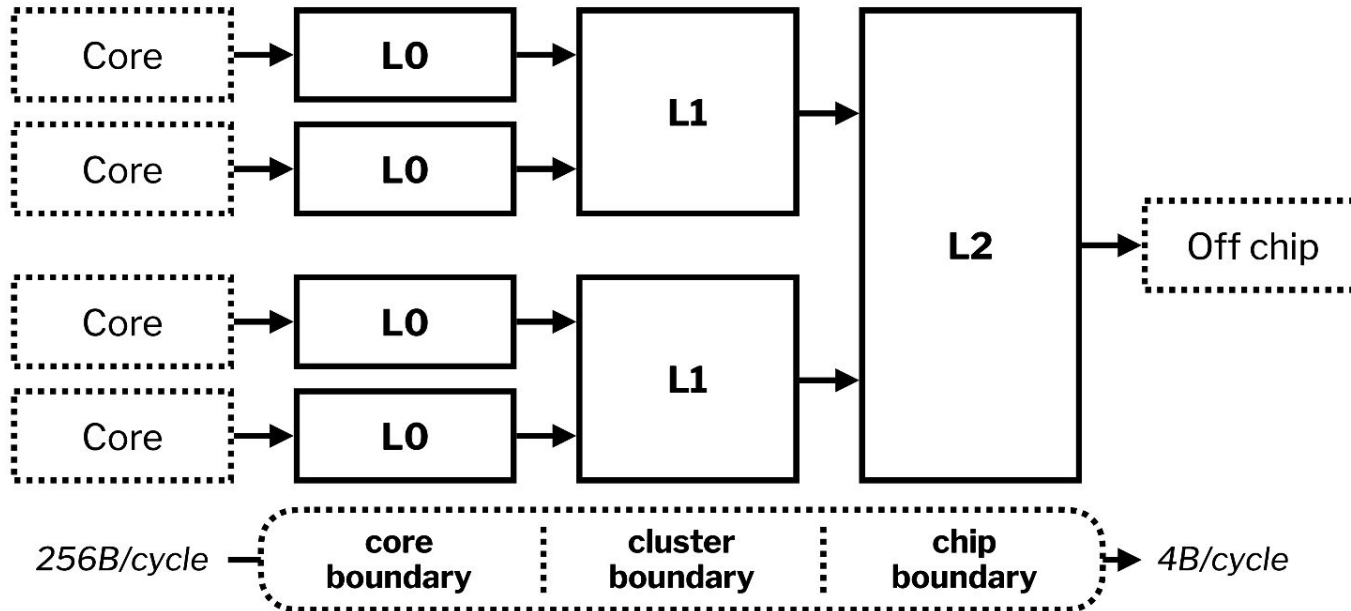
Objective: Bandwidth magnification from DRAM to SIMT cores



Global Memory



Objective: Bandwidth magnification from DRAM to SIMT cores



Solution

- Build cache hierarchy to match GPU topology
- Parameterize with latency & area considerations

Global Memory



L0i:

- 4KB DM
- 32B lines

L0d:

- 16KB DM
- 64B lines

L1:

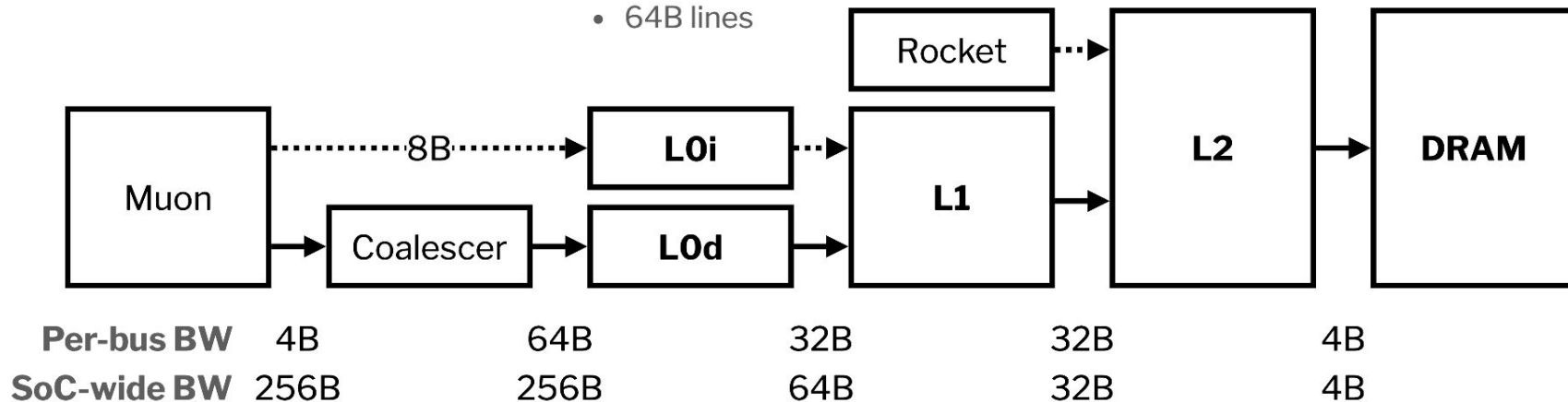
- 64KB
- 4 way SA
- 32B lines

L2:

- 512KB
- 8 way SA
- 32B lines

DRAM:

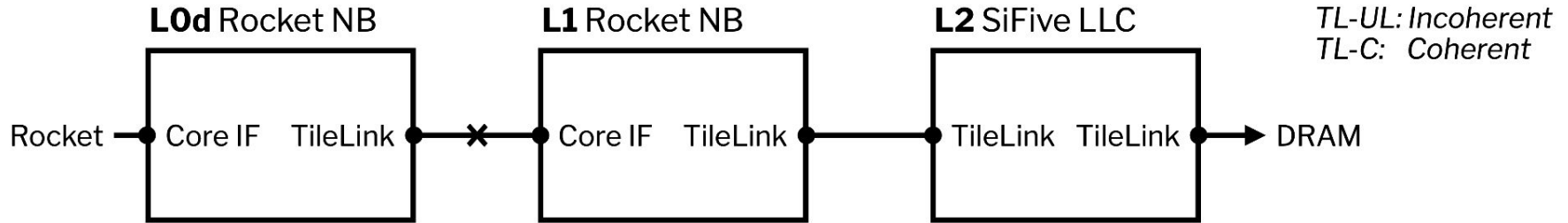
- 4GB CPU addressable
- 2GB GPU addressable



Global Memory



Objective: Composability & coherency

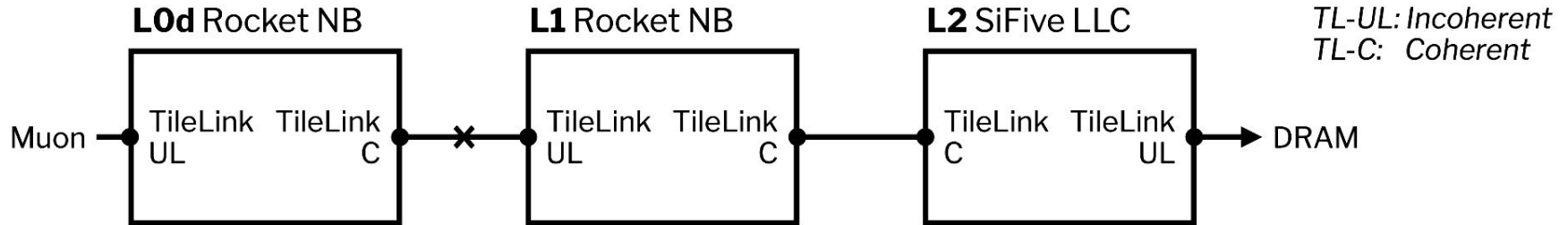


No composable caches in Chipyard

Global Memory



Objective: Composability & coherency



No composable caches in Chipyard

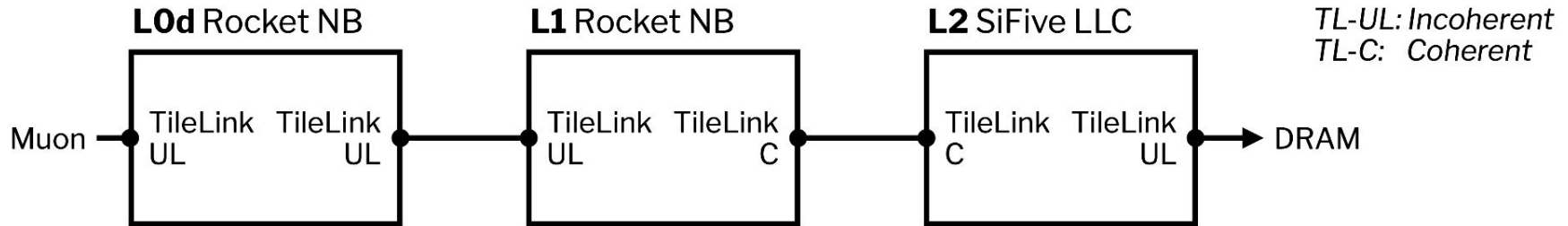
Core TileLink cannot support coherency

→ **Adapt core interface to TileLink**

Global Memory



Objective: Composability & coherency



No composable caches in Chipyard

Core TileLink cannot support coherency

L0d has no path to coherency agent

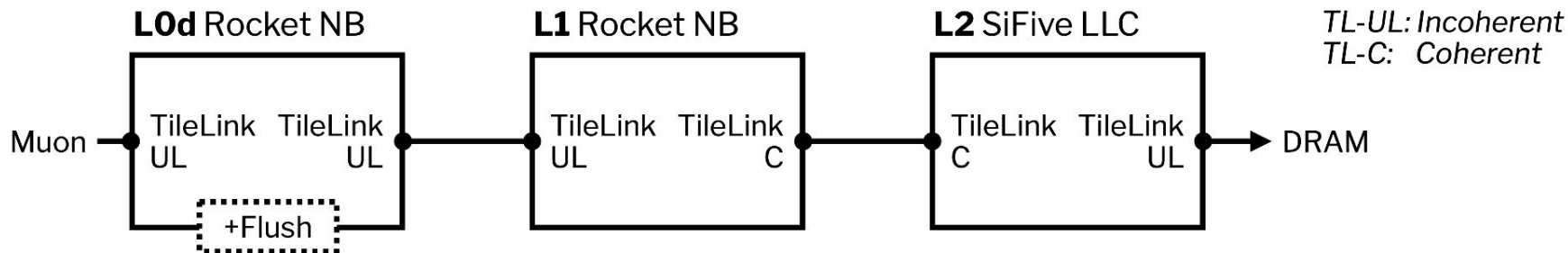
→ **Adapt core interface to TileLink**

→ **Map L0d coherent out traffic to incoherent**

Global Memory



Objective: Composability & coherency



No composable caches in Chipyard

Core TileLink cannot support coherency

L0d has no path to coherency agent

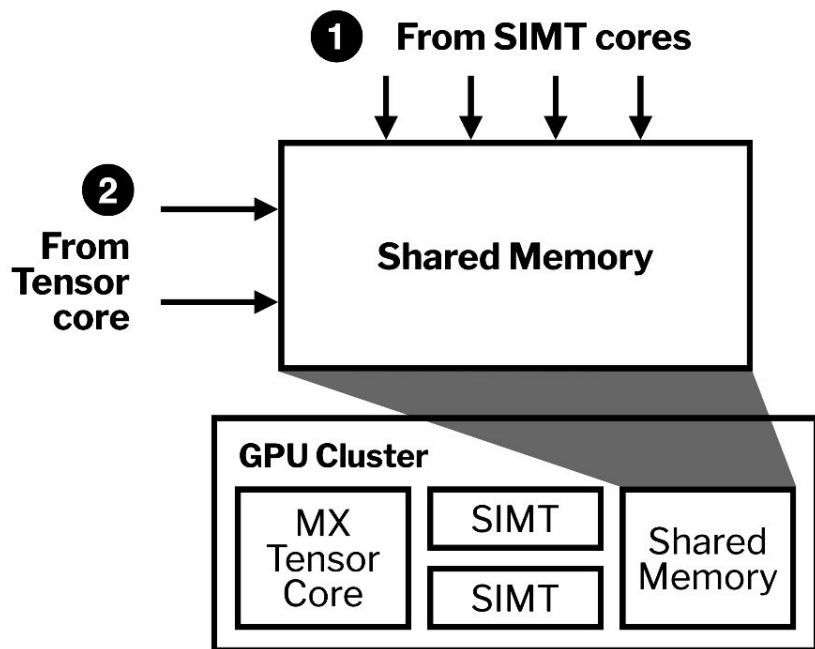
→ **Adapt core interface to TileLink**

→ **Map L0d coherent out traffic to incoherent**

→ **Add flushing unit for L0d fencing**

Cluster Shared Memory

Objective: Banking and interconnect strategy



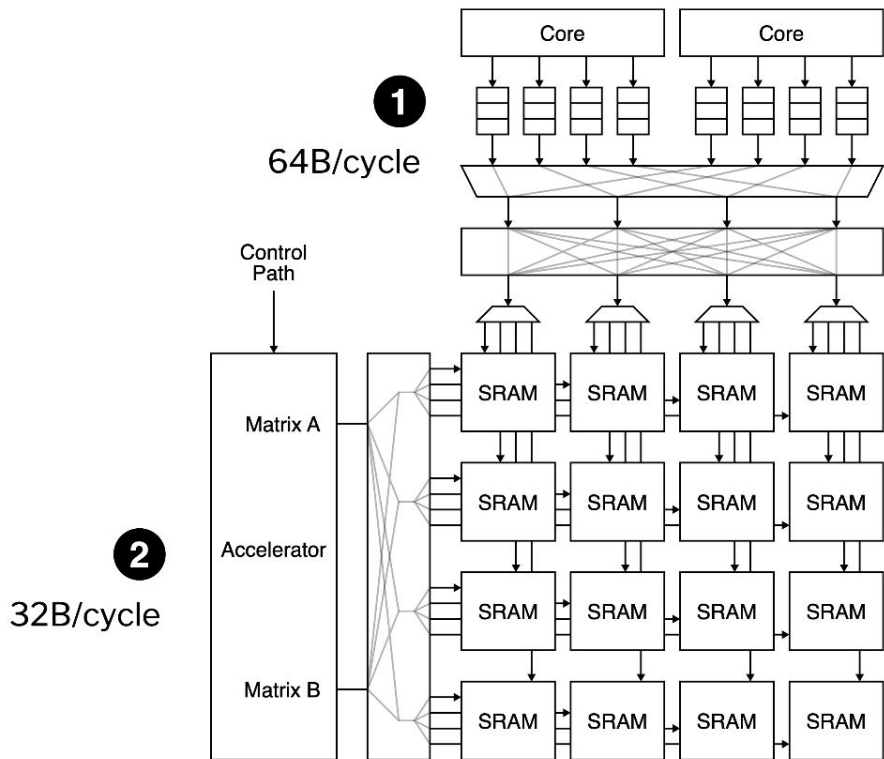
SIMT Access Patterns

- 1
- Contiguous during activations
 - Variable-stride during reductions
 - Random & broadcasted accesses

Tensor Access Patterns

- 2
- Contiguous, fixed-stride
 - Read-heavy
 - Software-pipelined across tiles, bank-parallel

Cluster Shared Memory



SIMT Core Clients

- Subbank-parallel ports
- Core-serialized
- Partial, random access

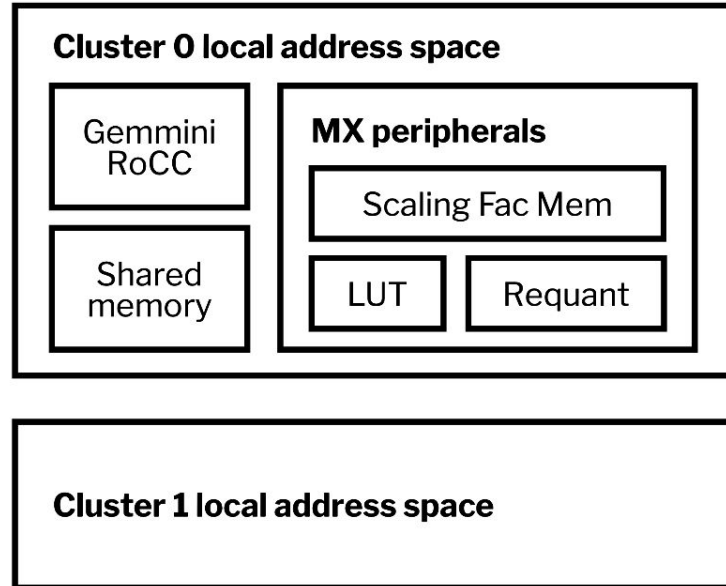
Tensor Core Clients

- Bank-parallel ports
- Bank-level crossbar
- Full-line access only

Cluster Shared Memory



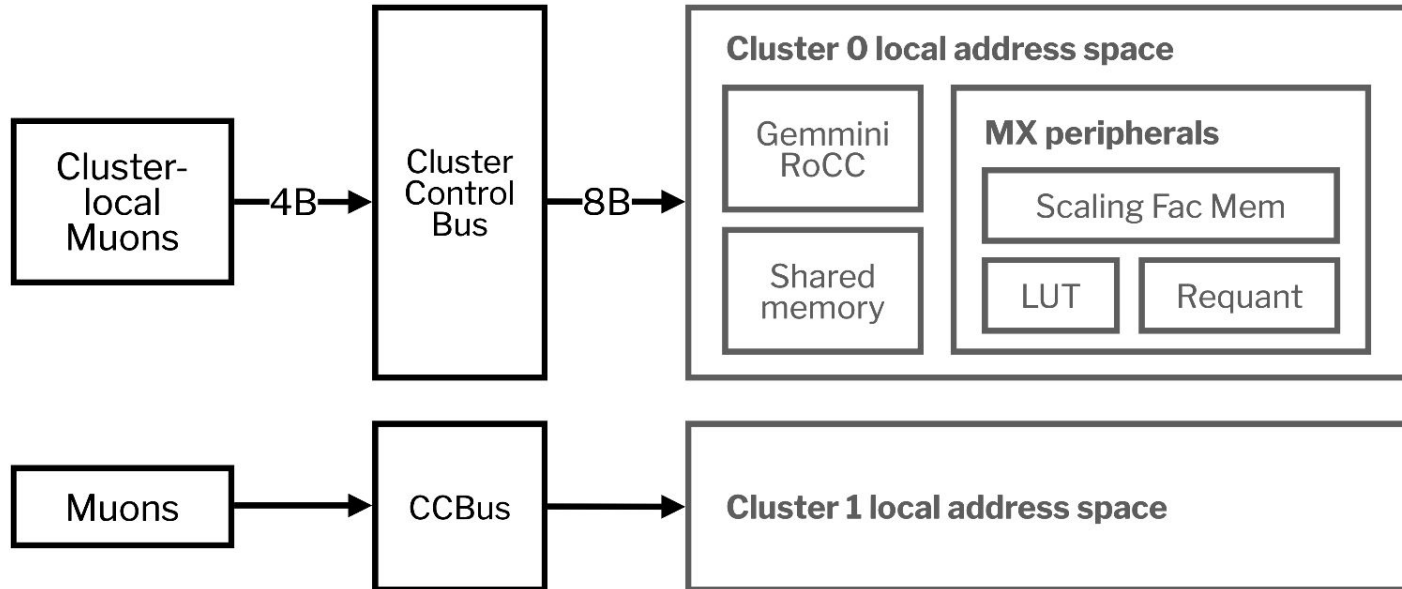
Objective: Flexible control over peripherals



Cluster Shared Memory



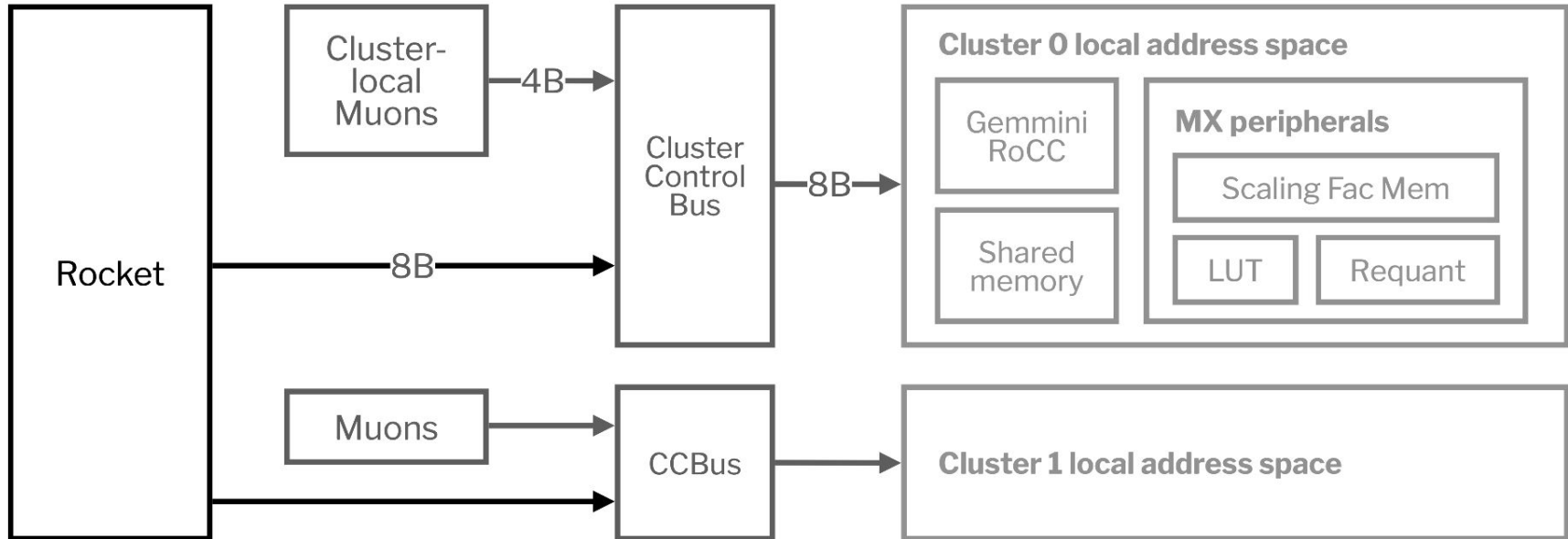
Objective: Flexible control over peripherals



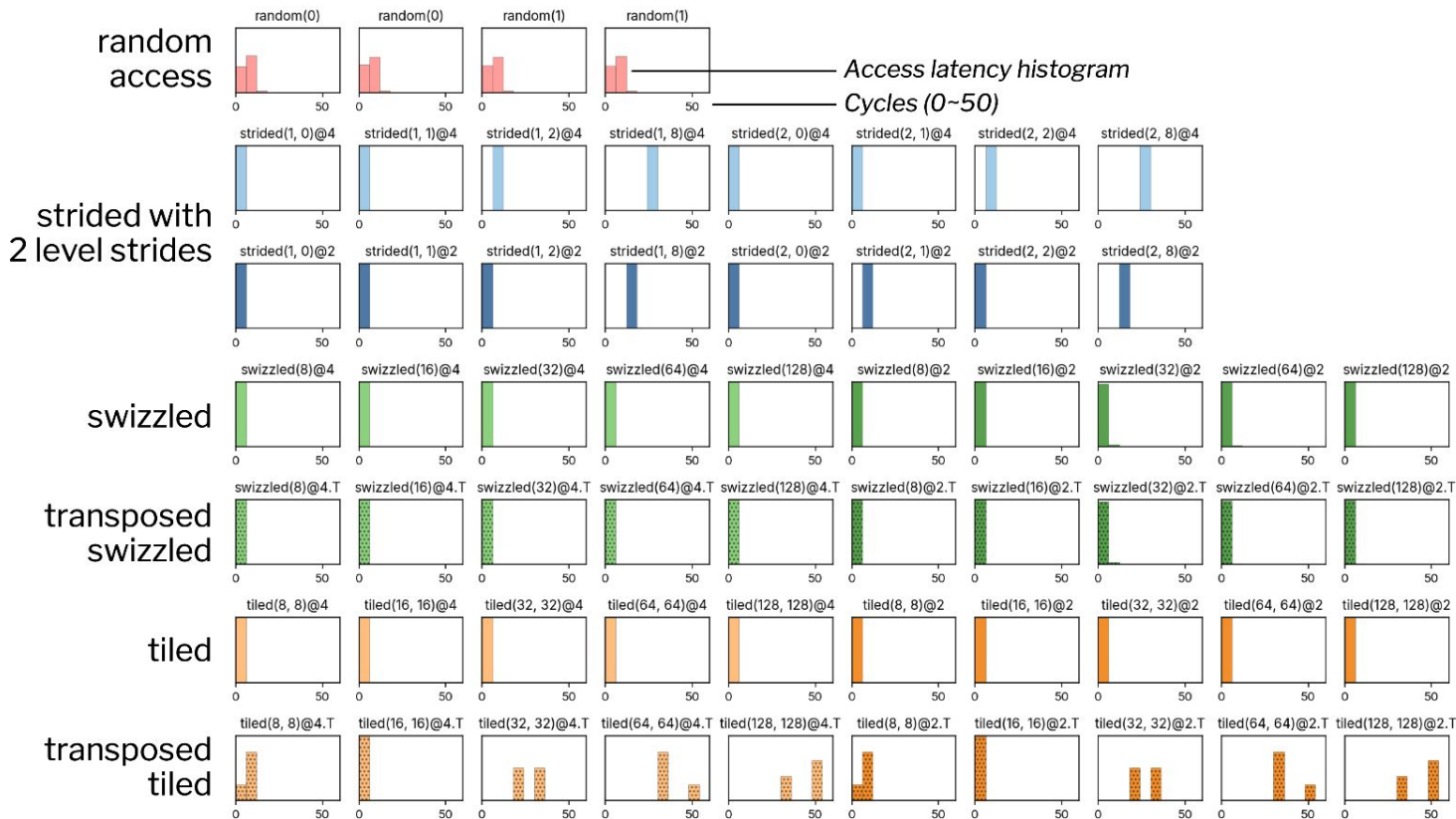
Cluster Shared Memory



Objective: Flexible control over peripherals



Cluster Shared Memory



Latency

- **≤5 cycles** for low bank-contention workloads
- **~10 cycles** for random accesses

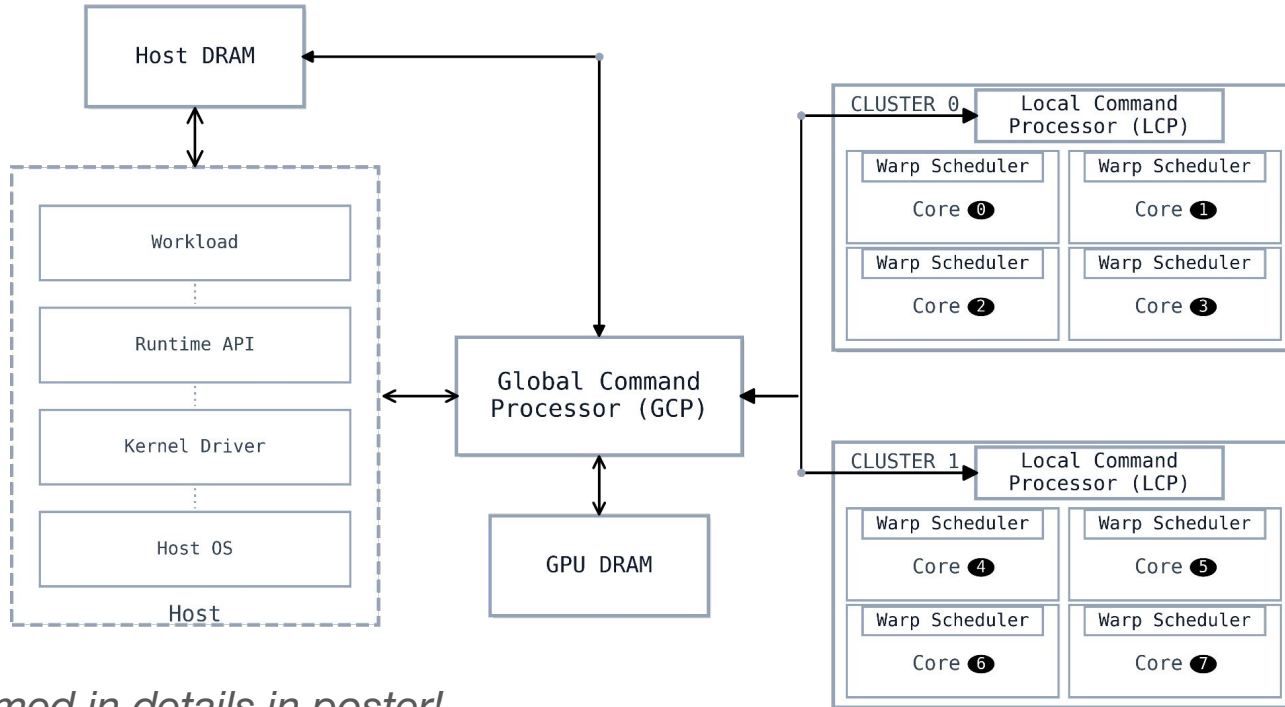
Throughput (not shown)

- **>90%** throughput for all strided, swizzled, non-transposed tiled
- **~60%** throughput for random



Command Processor

Command Processor



Zoomed in details in poster!



Command Processor

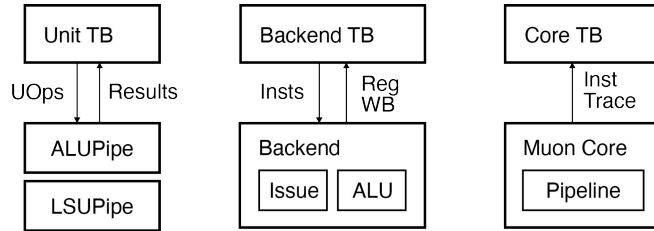
- **CUDA-like** host API
 - Designed for multitenancy: current iteration single address space
 - Multitenancy and virtual memory already possible with current API
 - Single command stream: API extensible to multiple streams
- **Hardware**
 - CMDs decoded and dispatched by **Global Command Processor (GCP)**
 - GCP computes & schedules threadblocks to **Local Command Processors (LCP)**
 - Explicit **printf ring** in device memory: printf unit to periodically drain ring
 - Kernel kill on fault: fault code, faulting thread, PC reported to host
- **Software**
 - Create 16 byte CMD header and assemble CMD payload
 - Memory and context management
 - Dependency management and enforcing CMD order



Execution & Conclusion

Multi-level Testbenches

Unit-top, Backend-top, Core-top



ISA / Perf Microbenchmarks

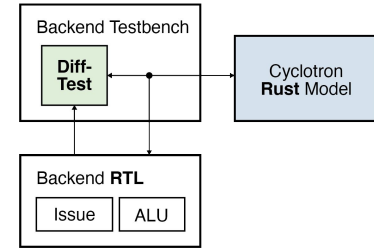
Targeted stress-tests
FLOPS/BW roofline checks

Cyclotron-RTL Co-verification

Isolating bugs

via co-sim

Ex: model frontend
+ RTL backend

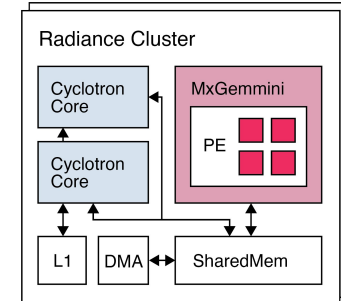


Diff-test

between golden
model and RTL

Progressive SoC

integration via
Cyclotron-as-a-Tile





Conclusion

Muon is a new open-source SIMT core in Radiance GPU to enable heterogeneous GPU research in academia with:

- Architectural support for modern kernel optimizations
- Parameterized & silicon-proven design on ASIC

More on...

- Microarchitecture / verification → *Poster*
- SoC-integration / infrastructure → *Poster*
- Command processor → *Poster*
- Tape-out with Gemini-Mx → *Amanda's talk*

